Ames Housing Price Prediction

1. Goal of the project

The goal is to predict the sale price of houses in Ames, Iowa from 79 descriptive features (size, quality, location, age, garage, basement and so on). The data contains **1,460 houses**. Because the price is a continuous number, this is a **regression** problem. The notebook follows three stages: understanding and exploring the data, preparing it for modelling, and building, tuning and comparing models. Every random step uses the same seed (42), so the results can be repeated.

2. Understanding the data

We first loaded the file and checked its shape and duplicates: there are 81 columns and no duplicated rows. Next we looked at missing values. At first sight 19 columns (7,829 cells) were incomplete, with pool, alley and fence columns almost empty. After checking the data description we found that in most of these columns a missing value does not mean "unknown", it means *the house does not have that feature*. So we filled them with the category "None" (or with 0 for veneer area, and the build year for garages without a year). Only *LotFrontage* (259 houses) and *Electrical* (1 house) were truly missing, and we left them to be filled later inside the modelling pipeline, so that no information leaks from the test data.

We then sorted the 79 predictors into **34 numerical, 23 nominal** (categories without order, such as neighbourhood) and **22 ordinal** features (categories with a natural order, such as quality ratings from Poor to Excellent) and wrote the correct order for every ordinal feature.

3. Exploring the data and main insights

The sale price is strongly right-skewed (skewness 1.88): a few very expensive houses stretch the distribution. After a log transformation the skewness drops to 0.12, so we decided to train the models on the log of the price. In total 18 of the 34 numeric features are also skewed, and we tested the Yeo-Johnson transformation, which reduced the skewness of all 18. We then looked for outliers and found two houses of 4,676 and 5,642 sq ft that sold for only $184,750 and $160,000. Both were incomplete ("Partial") sales, so their price does not reflect their size; we marked them as errors to remove. Genuine luxury homes were kept.

The correlation analysis shows that **overall quality** (r = 0.79) and **living area** (0.71) are the strongest predictors, followed by garage size and basement area. Among categorical features, **neighbourhood** explains 55% of the price variation. Nine focused questions gave clear answers: price rises steadily with quality (about $108k at quality 4 and $439k at quality 10), houses less than five years old cost almost twice as much as houses older than 60 years, more bathrooms mean a higher price, a basement adds about 73% on average, and the month of sale makes no significant difference.

4. Preparing the data

We removed the two anomalous houses (1,458 rows remain) and added four useful features: total bathrooms, house age, years since remodelling and total square footage. The data was split into **70% training, 15% validation and 15% test** (1,020 / 219 / 219 houses), keeping similar price distributions in each part. The test set is used only once, at the very end.

All cleaning steps are placed inside **pipelines** that learn from the training data only. Numeric features are filled with the median, the skewed ones are transformed with Yeo-Johnson and all are scaled with a robust scaler. Nominal features are filled with the most frequent value and one-hot encoded. Ordinal features use the order we defined. We compared five preprocessing combinations and kept Yeo-Johnson with the robust scaler because it gave the lowest error. The pipeline produces 242 model-ready columns, and we ran checks to prove that no information from validation or test data was used.

5. Building and tuning the models

We trained eight models that learn in different ways: Linear Regression, Ridge, Lasso, Elastic Net, KNN, SVR, Random Forest and Gradient Boosting. Each was first run with default settings and then tuned with 5-fold cross-validation on the training set. For SVR we searched kernels, C, gamma and epsilon in two stages (2,720 and then 750 fits); the best setting was an RBF kernel with C = 40, epsilon = 0.03 and gamma = 0.0005, which uses 710 support vectors (70% of

the training houses). We also used Optuna to tune an extra Random Forest. The best Lasso setting (alpha = 0.00046) keeps 103 of the 242 features and sets the other 139 to zero, so it also acts as an automatic feature selector.

Tuning improved the validation error by 21% on average, and most for Lasso and Elastic Net, whose default setting (alpha = 1) was too strong and made them underfit. Ridge, Lasso, Elastic Net and SVR fit well, with training and validation errors close together. Random Forest, Gradient Boosting and KNN overfit, because they are much more accurate on the training data than on new data. Plain Linear Regression was unstable on the validation set, so it is used only as a baseline. Elastic Net ended with l1_ratio = 1, which makes it identical to Lasso.

Model	Validation RMSE ($)	Validation R^2	Test RMSE ($)
Lasso (final)	**15,976**	**0.948**	**16,756**
Elastic Net	15,976	0.948	16,756
Ridge	16,087	0.948	17,265
SVR	15,229	0.953	16,117
Gradient Boosting	15,374	0.952	16,724
Random Forest	18,962	0.927	17,580
KNN	25,735	0.866	23,786

6. Final model and results

The final model was chosen using validation data only: among the models within 2% of the best validation error (Lasso, Elastic Net and Ridge) we take the simplest one, which is **Lasso**. On the untouched test set it reaches an **RMSE of $16,756, an MAE of $12,540 and R^2 of 0.928**. About 75% of test houses are predicted within 10% of their real price and 93% within 20%. A bootstrap analysis gives a 95% interval of $14,772 to $18,712 for the RMSE. The training, validation and test errors are close to each other, which shows that the model generalizes well. SVR is slightly more accurate in dollars (test RMSE $16,117) but it is just outside the 2% rule, much slower to tune and more complex. The final pipeline is saved as *final_model.joblib*. The main limits are the small test set (219 houses), the fact that the models run on the CPU, and that the data covers only one city and period.